AI Recommendations

1. Recommendation Engine (B2C)

    Suggest movies based on:

    User history

    Language preference

    Genre trends

    Tools:

    TensorFlow

    PyTorch

    Example:

    “Users who watched Action Hindi movies → recommend similar trending shows”

2. Dynamic Pricing (Revenue Optimization)

    AI adjusts ticket price based on:

    Demand

    Time (weekend vs weekday)

    Seat occupancy

    Techniques:

    Regression models

    Reinforcement learning

3.Fraud Detection (Payments)

    Detect suspicious transactions using:

    anomaly detection

    pattern recognition

4. Chatbot (Customer Support)

    Booking help, refunds, FAQs

    Tools:

    Dialogflow

    Rasa

5. Demand Prediction (B2B Insights)

    Help theatre partners:

    Predict occupancy

    Decide show timings

6. Personalized Notifications

    “Your favorite actor movie released”

    “Seats filling fast”

Architecture Fit

Microservices (Spring Boot)

Event-driven (Kafka)

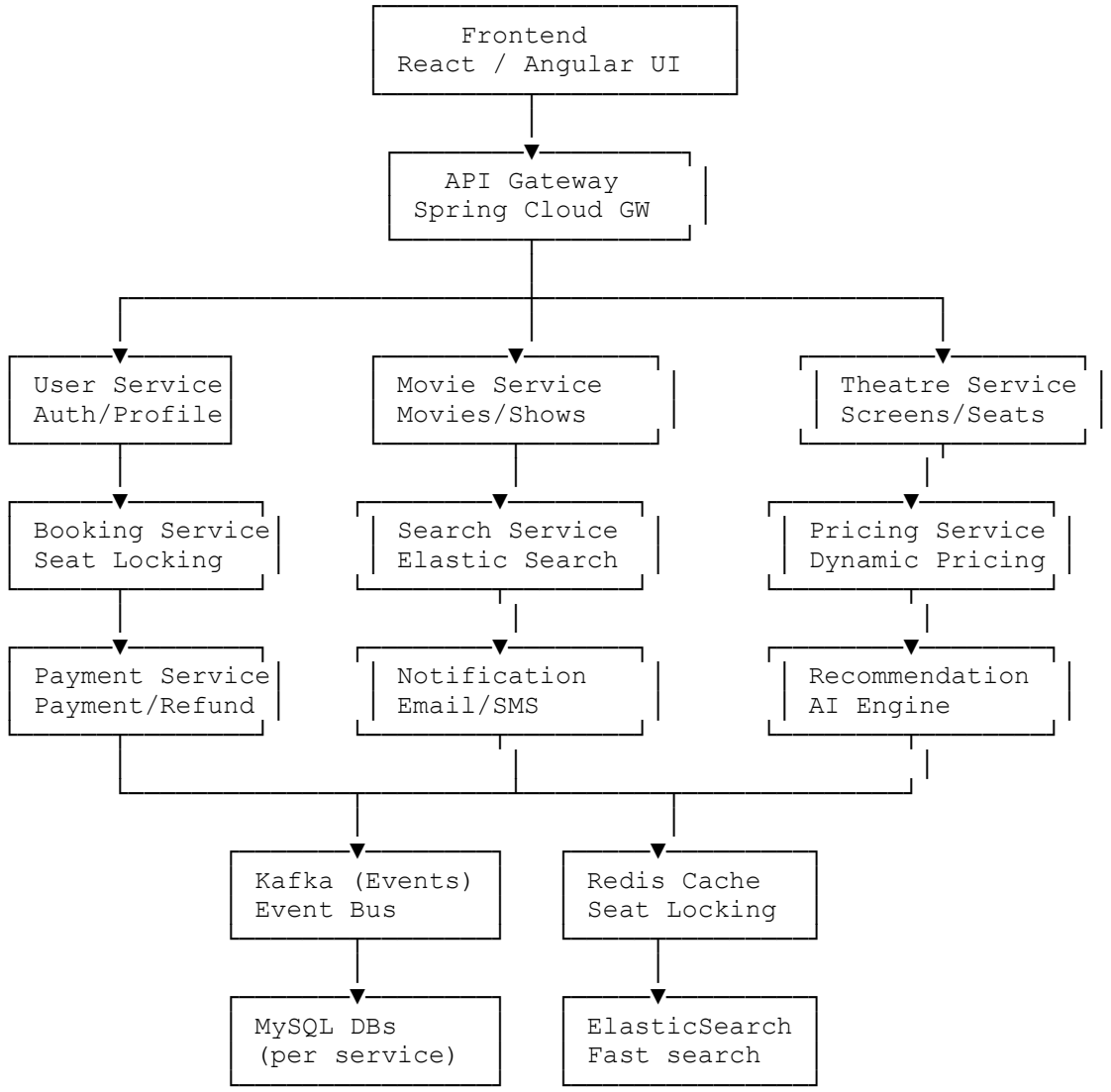
Cache-first (Redis for seat locking)

AI layer (recommendation + prediction)

TO BE Recommended Stack

Layer	Tech
Backend	Spring Boot + Spring Cloud
Frontend	React / Angular
DB	MySQL + Redis
Messaging	Kafka
Security	Keycloak
DevOps	Docker + Kubernetes
Monitoring	Prometheus + Grafana
AI	TensorFlow / PyTorch

////////////////
TO BE Architecture



////////////////
Microservices:
1. User Service

Login / Signup

JWT authentication

Roles: Admin, Partner, Customer

2. Movie Service

    Manage movies

    Show timings

    Language, genre filtering

3. Theatre Service (B2B)

    Theatre onboarding

    Screens, seats configuration

4. Search Service

    Fast search using ElasticSearch

    Filters:

    City

    Language

    Genre

5. Booking Service (CORE)

    Seat selection

    Seat locking (Redis)

    Booking lifecycle

6. Payment Service

    Mock / real payment gateway

    Refund handling

7. Pricing Service

    Dynamic pricing

    Peak vs off-peak logic

8. Notification Service

    Email / SMS

    Kafka consumers

9. Recommendation Service (AI)

    Personalized suggestions

    Trending movies

////////////////

Booking flow using Saga Pattern

Orchestration-based Saga (Booking Service controls flow)

Flow
1. User selects seats
2. Booking Service → Lock seats (Redis)
3. Booking Service → Create booking (PENDING)

4. Booking Service → Call Payment Service
5. Payment Success → Confirm booking
6. Payment Failure → Release seats

Saga Steps

Step	Action	Compensation
1	Lock Seats	Unlock seats
2	Create Booking	Cancel booking
3	Payment Refund / rollback	
4	Confirm Booking End	

////////////////
Sequence Diagram (Booking)
User → API Gateway → Booking Service

Booking Service → Redis → Lock Seats
Booking Service → DB → Save PENDING booking

Booking Service → Payment Service → Process Payment

IF SUCCESS:
    Booking Service → DB → CONFIRMED
    Booking Service → Kafka → BookingConfirmedEvent
    Notification Service → Send Ticket

IF FAILURE:
    Booking Service → Redis → Release Seats
    Booking Service → DB → CANCELLED
////////////////
Using Redis for seat locking is one of the most critical design decisions in a movie booking system — it prevents double booking under high concurrency.

Problem:

100 users try to book same seat (A1) at the same time

Without locking → multiple bookings succeed

Solution:

Use Redis atomic operations → only ONE user can lock the seat

How It Works
User selects seat → Try LOCK in Redis
    ↓
If LOCK SUCCESS → proceed booking
If LOCK FAIL → seat already taken

Redis Strategy
Use SETNX (Set if Not Exists)

Atomic operation
Ensures only one request succeeds

Redis Key Design
Key: showId:seatNumber
Example: SHOW123:A1
Value: LOCKED_BY_USER_101
TTL: 5 minutes

Why TTL

If user doesn’t complete payment → lock auto expires
Prevents deadlocks

Spring Boot Implementation
1. Lock Seat
```
public boolean lockSeat(String showId, String seat, String userId) {
```

```
    String key = showId + ":" + seat;

    Boolean success = redisTemplate.opsForValue()
        .setIfAbsent(key, userId, Duration.ofMinutes(5));

    return Boolean.TRUE.equals(success);
}

2. Unlock Seat (on failure)
public void unlockSeat(String showId, String seat) {
    String key = showId + ":" + seat;
    redisTemplate.delete(key);
}

3. Validate Lock Owner (Important)
public boolean isSeatLockedByUser(String showId, String seat, String userId) {
    String key = showId + ":" + seat;
    String value = redisTemplate.opsForValue().get(key);
    return userId.equals(value);
}

Booking Flow with Redis
1. User selects seats
2. Try lock seats in Redis
3. If ANY lock fails → abort booking
4. If all success → proceed to payment
5. Payment success → confirm booking (DB)
6. Payment failure → release locks


Important Edge Cases
1. Partial Lock Failure

Seat A1 locked, A2 failed
    Rollback A1

2. Payment Timeout
    TTL auto releases seat

3. Double Click / Retry
    Redis prevents duplicate locks

4. Service Crash
    TTL ensures recovery

//////////////
3. Avoid DB Locking

Bad:

SELECT ... FOR UPDATE

Causes:
Deadlocks
Poor scalability

Redis vs DB Locking
Feature Redis   DB
Speed   Very fast   Slow
Scalability High    Limited
Deadlocks   No  Yes
TTL support Yes No
//////////////////////
How TTL Ensures Recovery After Service Crash
Problem Without TTL

Imagine this flow:

1. User selects seat A1
```

2. Booking Service locks seat in Redis
3. Service crashes BEFORE payment completes

Result:

Seat remains LOCKED forever

No one else can book it

System gets stuck / inconsistent

How TTL Fixes This

When you lock a seat, you set a TTL (Time To Live):

```
redisTemplate.opsForValue()
    .setIfAbsent(key, userId, Duration.ofMinutes(5));
```

What Happens Internally
Seat LOCK created → TTL = 5 minutes

Case: Service crashes
    ↓
Redis timer keeps running
    ↓
After 5 minutes → key auto deleted
    ↓
Seat becomes AVAILABLE again

Why This Works Even If Service Dies
Redis runs independently (separate process/container)
TTL countdown is handled inside Redis, not your app
No need for manual cleanup

Timeline Example
10:00 → Seat A1 locked (TTL 5 min)
10:01 → Service crashes
10:05 → TTL expires
10:05 → Redis deletes key
10:05 → Seat A1 is free again

Important Design Insight

TTL acts like a “safety net”

Even if:

App crashes

Network fails

Payment service is down
System self-heals automatically
Movie booking apps release seats if payment not done

What If TTL Is NOT Used?

Scenario	Result
Service crash	Seats locked forever
Payment failure	Manual cleanup needed
High traffic	System unusable

Best Practices
Choose Correct TTL
Usually 2–10 minutes
Balance:
Too short → user frustration
Too long → poor availability

Extend TTL (Optional Advanced)
If payment is in progress:

```
redisTemplate.expire(key, Duration.ofMinutes(5));
Store User Info in Lock
Value: userId

Helps validate ownership before confirming booking

Use Unique Booking ID
Value: bookingId:userId

TTL ensures recovery by automatically releasing seat locks if the booking service
crashes or fails mid-flow. Since Redis handles expiration independently, it prevents
deadlocks and keeps the system consistent without manual intervention.


TTL + Saga Pattern
    TTL complements Saga:
    Saga handles logical rollback
    TTL handles unexpected failures


////////////////
Redis + Kafka + Saga fully wired project

Redis (seat locking)
Kafka (event-driven flow)
Saga (orchestration)
Spring Boot microservices
Docker Compose (run everything)


1. Project Structure
movie-booking-platform/
│
├── docker-compose.yml
│
├── booking-service/
├── payment-service/
├── notification-service/
│
└── common-lib/ (DTOs, events)

2. Core Flow (End-to-End)
User → Booking Service
   ↓
Redis → Lock Seat
   ↓
Booking → PENDING
   ↓
Kafka → payment-request
   ↓
Payment Service → process
   ↓
Kafka → payment-success / payment-failed
   ↓
Booking Service (Saga)
   ↓
CONFIRM / CANCEL
   ↓
Kafka → notification


3. Saga (Orchestration with Kafka)

Booking Service = Saga Orchestrator

Events:
payment-request
payment-success
```

```
payment-failed
booking-confirmed
booking-cancelled


4. Booking Service (CORE)
4.1 Entity
@Entity
@Data
public class Booking {
    @Id
    @GeneratedValue
    private Long id;

    private String showId;
    private String seat;
    private String status; // PENDING, CONFIRMED, CANCELLED
}
4.2 Redis Seat Lock
@Service
@RequiredArgsConstructor
public class SeatLockService {

    private final RedisTemplate<String, String> redisTemplate;

    public boolean lockSeat(String showId, String seat, String userId) {
        String key = showId + ":" + seat;

        return Boolean.TRUE.equals(
            redisTemplate.opsForValue()
                .setIfAbsent(key, userId, Duration.ofMinutes(5))
        );
    }

    public void unlockSeat(String showId, String seat) {
        redisTemplate.delete(showId + ":" + seat);
    }
}
4.3 Kafka Producer (Trigger Payment)
@Service
@RequiredArgsConstructor
public class BookingProducer {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void sendPaymentRequest(Booking booking) {
        kafkaTemplate.send("payment-request", booking);
    }
}
4.4 Booking Service (Saga Orchestrator)
@Service
@RequiredArgsConstructor
public class BookingService {

    private final BookingRepository repo;
    private final SeatLockService seatLockService;
    private final BookingProducer producer;

    public Booking createBooking(String showId, String seat, String userId) {

        // Step 1: Lock seat
        boolean locked = seatLockService.lockSeat(showId, seat, userId);

        if (!locked) {
            throw new RuntimeException("Seat already booked");
        }

        // Step 2: Save booking
        Booking booking = new Booking();
```

```
        booking.setShowId(showId);
        booking.setSeat(seat);
        booking.setStatus("PENDING");
        repo.save(booking);

        // Step 3: Trigger payment
        producer.sendPaymentRequest(booking);

        return booking;
    }
}
```

4.5 Kafka Consumer (Handle Saga)

```
@Component
@RequiredArgsConstructor
public class PaymentConsumer {

    private final BookingRepository repo;
    private final SeatLockService seatLockService;

    @KafkaListener(topics = "payment-success")
    public void handleSuccess(Booking booking) {
        booking.setStatus("CONFIRMED");
        repo.save(booking);
    }

    @KafkaListener(topics = "payment-failed")
    public void handleFailure(Booking booking) {

        // Compensation
        seatLockService.unlockSeat(booking.getShowId(), booking.getSeat());

        booking.setStatus("CANCELLED");
        repo.save(booking);
    }
}
```

5. Payment Service

Kafka Consumer + Producer

```
@Component
@RequiredArgsConstructor
public class PaymentProcessor {

    private final KafkaTemplate<String, Object> kafkaTemplate;

    @KafkaListener(topics = "payment-request")
    public void processPayment(Booking booking) {

        boolean success = Math.random() > 0.3;

        if (success) {
            kafkaTemplate.send("payment-success", booking);
        } else {
            kafkaTemplate.send("payment-failed", booking);
        }
    }
}
```

6. Notification Service

```
@Component
public class NotificationConsumer {

    @KafkaListener(topics = "payment-success")
    public void sendTicket(Booking booking) {
        System.out.println("Ticket sent for booking " + booking.getId());
    }
}
```

7. Kafka Config (Common)

```
@Configuration
public class KafkaConfig {
```

```
    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> config = new HashMap<>();
        config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092");
        config.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
        return new DefaultKafkaProducerFactory<>(config);
    }

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return new KafkaTemplate<>(producerFactory());
    }
}
```

8. Docker Compose

```
version: '3.8'

services:

  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3306:3306"

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092

  booking-service:
    build: ./booking-service
    ports:
      - "8081:8081"
    depends_on:
      - mysql
      - redis
      - kafka

  payment-service:
    build: ./payment-service
    ports:
      - "8082:8082"
    depends_on:
      - kafka

  notification-service:
    build: ./notification-service
    depends_on:
      - kafka
```

```
/////////////////

Resilience4j (retry + circuit breaker)

Adding Resilience4j (Retry + Circuit Breaker) makes  system handling failures gracefully (e.g., Payment Service downtime).

Payment service is:

Slow

Down

Intermittent

Without protection:

Threads block

System crashes under load


Solution

Retry → try again (temporary failures)

Circuit Breaker → stop calling failing service


Flow with Resilience4j
Booking Service → Payment Service

IF failure:
   Retry (3 times)
   ↓
IF still failing:
   Circuit opens
   ↓
Fallback → Cancel booking + release seat
⚙️1. Add Dependency (Maven)
<dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
2. Configuration (application.yml)
resilience4j:

  retry:
    instances:
      paymentRetry:
        max-attempts: 3
        wait-duration: 2s

  circuitbreaker:
    instances:
      paymentCB:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
3. Payment Client (Feign / RestTemplate)
With Resilience4j Annotations
@Service
public class PaymentClient {

    @Retry(name = "paymentRetry")
    @CircuitBreaker(name = "paymentCB", fallbackMethod = "fallback")
    public boolean pay(Long bookingId) {

        // call payment service
```

```
        ResponseEntity<Boolean> response =
            restTemplate.postForEntity(
                "http://payment-service/payment/" + bookingId,
                null,
                Boolean.class
            );

        return response.getBody();
    }

    // Fallback method
    public boolean fallback(Long bookingId, Exception ex) {
        System.out.println("Payment failed, triggering fallback");

        return false;
    }
}

4. Integrate with Saga (Booking Service)
boolean success = paymentClient.pay(booking.getId());

if (success) {
    booking.setStatus("CONFIRMED");
} else {
    // Compensation
    seatLockService.unlockSeat(showId, seat);
    booking.setStatus("CANCELLED");
}

5. Circuit Breaker States
CLOSED → Normal calls
OPEN → Calls blocked (fallback only)
HALF-OPEN → Test calls

Case: Payment Down
1st call → fail
2nd call → retry → fail
3rd call → retry → fail

→ Circuit opens
→ Next calls → NO request sent
→ Direct fallback executed

6. Advanced (Recommended)
Add Timeout
resilience4j:
  timelimiter:
    instances:
      paymentTimeout:
        timeout-duration: 3s

Combine All
@Retry(name = "paymentRetry")
@CircuitBreaker(name = "paymentCB", fallbackMethod = "fallback")
@TimeLimiter(name = "paymentTimeout")

7. Monitoring (Very Important)

Expose metrics:

/actuator/health

/actuator/metrics

/actuator/circuitbreakers

Integrate with:

Prometheus
```

Grafana

⚠ Best Practices
Retry Only for:

Network failures

Temporary issues

Avoid Retry for:

Business failures (payment declined)

Keep Retry Small

max 2-3 attempts

Use Fallback Smartly

Release seat

Cancel booking

Notify user

Resilience4j + Saga
Component Responsibility
Retry Temporary recovery
Circuit Breaker System protection
Saga Data consistency
/////////////////////////